

Zadanie H – Rozwijalna lista

Punktów do uzyskania: 9

Generalia

- Zadnie polega na implementacji obsługi ciągów danych zewnętrznie zdefiniowanego typu OBJECT_TYPE, zwanych odtąd *obiektami*.
- Obiekty są pamiętane w dynamicznie przydzielanych tablicach.
- Wskaźniki do tablic obiektów są polami struktury nazywanej odtąd *węzłem*.
- Węzły są powiązane w implementowaną wskaźnikowo listę pojedynczo wiązaną.
- Węzły definiuje zewnętrzna struktura NODE_STRUCT postaci:

```
struct NODE_STRUCT {
    OBJECT_TYPE* object;
    BYTE use;
    NODE_STRUCT* next;
}
```

- ... gdzie pole:
- **object** - wskazuje tablicę obiektów.
 - **use** - pamięta ilość aktualnie używanych obiektów.
 - **next** - wskazuje następny węzeł.
- Zdefiniowany zewnętrznie typ BYTE jest równoważny typowi **unsigned char**.
 - Długość przydzielanej tablicy obiektów jest zawsze stała i określona zdefiniowaną zewnętrznie wartością SIZE typu BYTE.
 - Używane obiekty zajmują zawsze spójny początkowy fragment tablicy.
 - Lista węzłów nie może zawierać tablic bez używanych obiektów.
 - Użyte dalej sformułowanie określenie obiektu oznacza określenie węzła z tablicą obiektów oraz indeks obiektu w zawierającej obiekt tablicy.**
 - Dla obiektów dostępne są operatory równości i różności.

- ### Działania obsługi pamięci
- NODE_STRUCT* NewNode (void)
 - Tworzy nowy węzeł listy typu NODE_STRUCT.
 - Przydziela tablicę obiektów typu OBJECT_TYPE do pola object.
 - Zwraca wskaźnik do utworzonego węzła.
 - Stanowi jedyny podprogram dopuszczający użycie operatora **new**.
 - Stanowi jeden z dwóch podprogramów dopuszczających użycie znaków prostokątnych nawiasów.
 - void** DeleteNode (NODE_STRUCT*)
 - Zwraca pamięć węzła danego argumentem, wraz ze zwróceniem pamięci tablicy obiektów.
 - Stanowi jedyny podprogram dopuszczający użycie operatora **delete**.
 - Stanowi jeden z dwóch podprogramów dopuszczających użycie znaków prostokątnych nawiasów.

- void** Clear (NODE_STRUCT**)
 - Usuwa listę węzłów przekazaną argumentem zwracając używaną pamięć węzłów i pobranych tablic obiektów.
 - Listę węzłów przekazaną argumentem ustawia na wartość NULL.

- ### Działania dodające obiekty
- Dla działań dodających obiekty obowiązują reguły:
 - Jeżeli dodanie obiektu jest możliwe poprzez zwiększenie ilości używanych obiektów w istniejącym węźle należy je wykonać w pierwszej kolejności.
 - Jeżeli dodawanie obiektu w istniejącym węźle nie jest możliwe i konieczne jest stworzenie nowego węzła z jednym obiektem.
 - void** AddFirst (NODE_STRUCT**, OBJECT_TYPE*)
 - Dodaje obiekt na początek ciągu.
 - Pierwszy argument wskazuje listę węzłów.
 - Drugi argument wskazuje obiekt do dodania.
 - Ewentualny nowy węzeł może być dodany wyłącznie na początek listy węzłów.
 - void** AddLast (NODE_STRUCT**, OBJECT_TYPE*)
 - Dodaje obiekt na koniec ciągu.
 - Pierwszy argument wskazuje listę węzłów.
 - Drugi argument wskazuje obiekt do dodania.
 - Ewentualny nowy węzeł może być dodany wyłącznie na koniec listy węzłów.

- ### Działania pobierające obiekty
- void** GetFirst (NODE_STRUCT*, NODE_STRUCT**, BYTE*)
 - Podaje określenie pierwszego obiektu ciągu.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci przyjmują określenie pierwszego obiektu.
 - Dla pustej listy węzłów drugi argument przyjmuje wartość NULL, zaś zmienna przekazana trzecim argumentem pozostaje bez zmian.
 - void** GetPrev (NODE_STRUCT*, NODE_STRUCT*, BYTE, NODE_STRUCT**, BYTE*)
 - Podaje obiekt poprzedzający względem wskazanego obiektu.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci określają wskazany obiekt w ciągu.
 - Argumenty czwarty i piąty przyjmują określenie poprzedzającego obiektu.
 - Gdy poprzedzającego obiektu brak, czwarty argument przyjmuje wartość NULL, zaś zmienna przekazana piątym argumentem pozostaje bez zmian.
 - void** GetNext (NODE_STRUCT*, NODE_STRUCT*, BYTE, NODE_STRUCT**, BYTE*)
 - Podaje obiekt następujący względem wskazanego obiektu.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci określają wskazany obiekt w ciągu.
 - Argumenty czwarty i piąty przyjmują określenie następującego obiektu.
 - Gdy następującego obiektu brak, czwarty argument przyjmuje wartość NULL, zaś zmienna przekazana piątym argumentem pozostaje bez zmian.

- void** GetLast (NODE_STRUCT*, NODE_STRUCT**, BYTE*)
 - Podaje określenie ostatniego obiektu ciągu.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci przyjmują określenie ostatniego obiektu.
 - Dla pustej listy węzłów drugi argument przyjmuje wartość NULL, nie zmieniając wartości przekazanej trzecim argumentem.

- ### Działania wstawiające obiekty
- Działania wstawiające podlegają regułom działań dodających.
 - void** InsertPrev (NODE_STRUCT**, NODE_STRUCT*, BYTE, OBJECT_TYPE*)
 - Wstawia obiekt przed wskazanym obiektem.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci określają wskazany obiekt w ciągu.
 - Czwarty argument wskazuje wstawiany obiekt.
 - Nowy węzeł może być dodany wyłącznie przed węzłem przekazanym drugim argumentem.
 - void** InsertNext (NODE_STRUCT*, NODE_STRUCT*, BYTE, OBJECT_TYPE*)
 - Wstawia obiekt za wskazanym obiektem.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci określają wskazany obiekt w ciągu.
 - Czwarty argument wskazuje wstawiany obiekt.
 - Nowy węzeł może być dodany wyłącznie za węzłem przekazanym drugim argumentem.

- ### Działania usuwające obiekty
- Dla działań usuwających obowiązują poniższe reguły.
 - Wszystkie działania usuwające w pierwszej kolejności usuwają obiekt poprzez zmniejszenie ilości używanych obiektów w tablicy węzła.
 - Węzły osiągające zerową ilość używanych obiektów są usuwane.
 - void** RemoveFirst (NODE_STRUCT**)
 - Usuwa pierwszy obiekt ciągu.
 - Argument wskazuje na listę węzłów.
 - Dla pustej listy nie jest wykonywane żadne działanie.
 - void** RemovePrev (NODE_STRUCT**, NODE_STRUCT*, BYTE)
 - Usuwa z ciągu obiekt przed określonym obiektem.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci określają obiekt.
 - W przypadku braku obiektu poprzedzającego nie jest wykonywane żadne działanie.

- void** RemoveCurrent (NODE_STRUCT**, NODE_STRUCT*, BYTE)
 - Usuwa z ciągu określony obiekt.
 - Pierwszy argument wskazuje listę węzłów.
 - Argumenty drugi i trzeci określają obiekt do usunięcia.
- void** RemoveNext (NODE_STRUCT*, NODE_STRUCT*, BYTE)
 - Usuwa obiekt za określonym obiektem.
 - Pierwszy argument przekazuje listę węzłów.
 - Argumenty drugi i trzeci określają obiekt.
 - W przypadku braku obiektu następującego nie jest wykonywane żadne działanie.
- void** RemoveLast (NODE_STRUCT**)
 - Usuwa ostatni obiekt ciągu.
 - Argument wskazuje na listę węzłów.
 - Dla pustej listy nie jest wykonywane żadne działanie.

- ### Działania dodatkowe
- void** Find (NODE_STRUCT*, OBJECT_TYPE*, NODE_STRUCT**, BYTE*)
 - Wyszukuje określenie pierwszego obiektu równego wskazanemu obiektowi.
 - Pierwszy argument przekazuje listę węzłów.
 - Drugi argument wskazuje obiekt do wyszukania.
 - Argumenty trzeci i czwarty przyjmują określenie wyszukiwanego obiektu.
 - Jeżeli wyszukiwanego obiektu brak, trzeci argument przekazuje wartość NULL, zaś zmienna przekazana czwartym argumentem pozostaje bez zmian.

- void** Compress (NODE_STRUCT*)

Kompresuje listę węzłów daną argumentem do postaci z wszystkimi węzłami mającymi pełne tablice obiektów z wyjątkiem ostatniego.
- void** Reverse (NODE_STRUCT*)

Odwraca ciąg obiektów i kompresuje listę węzłów daną argumentem.

- ### Ogólne warunki rozwiązania
- Rozwiązanie dopuszcza wyłącznie wiedzę przedstawioną na wykładzie.
 - Zabronione jest włączanie jakichkolwiek plików nagłówkowych.
 - Używanie znaków kwadratowych nawiasów [] jest dopuszczalne wyłącznie w podprogramach NewNode oraz DeleteNode.
 - W szczególności zabronione jest używanie:
 - Standardowych kontenerów sekwencyjnych w rodzaju array, list, queue, vector czy forward_list.
 - Sekwencji string oraz **class**.
 - Zmiennych globalnych.
 - Własnych podprogramów rozpoczynających się znakiem podkreślenia.
 - Każdy test na BaCy występuje w dwóch wersjach.
 - Wersja zaczynająca się słowem not, sprawdza prawidłowość działania operacji ciągowych, ale nie sygnalizuje błędów obsługi pamięci. Nie kontroluje wycieków pamięci ani błędnego użycia operatora **delete** i nie przydziela punktów.
 - Docelowa wersja zaczyna się słowem chk. Posługuje się jednakowym wejściem i wyjściem względem wersji ze słowem not, ale kontroluje wycieki pamięci oraz poprawne stosowanie operatora **delete**. Zarazem przydziela punkty za zadanie w przypadku poprawnego rozwiązania.

- ### Wymagania dotyczące pamięci dynamicznej
- Operator **new** może być użyty wyłącznie w podprogramie NewNode.
 - Operator **delete** może być użyty wyłącznie w podprogramie DeleteNode.
 - Użyta pamięć dynamiczna musi być zwracana.
 - Całkowicie dyskwalifikują rozwiązanie:
 - Wycieki pamięci.
 - Użycie operatora **delete** dla wartości wynoszących NULL lub **nullptr**.